

Tkinter

1. What is Tkinter?

Tkinter is a built-in Python library that allows us to create **Graphical User Interfaces (GUIs)** such as windows, buttons, text boxes, and more.

You can make apps like calculators, forms, and notepads using Tkinter.

To use Tkinter, you simply import it:

```
import tkinter as tk
```

2. Creating Your First Window

Every Tkinter program has three main steps:

1. Create a window
2. Set its title and size
3. Keep it running using `mainloop()`

```
import tkinter as tk
```

```
root = tk.Tk()          # Step 1: Create main window
```

```
root.title("My First App") # Step 2: Set title
```

```
root.geometry("300x200")  # Optional: Set size (width x height)
```

```
root.mainloop()         # Step 3: Display window until closed
```

Explanation:

- `Tk()` creates the main application window.
 - `title()` sets the window name.
 - `geometry()` changes window size.
 - `mainloop()` keeps the window active until the user closes it.
-

3. Adding a Label (Text on Screen)

Labels display text or images inside the window.

```
import tkinter as tk
```

```
root = tk.Tk()

root.title("Label Example")


label = tk.Label(root, text="Welcome to Tkinter!")

label.pack() # Adds label to the window


root.mainloop()
```

Explanation:

- `Label()` creates a label widget.
 - `pack()` adds it to the window (places it automatically).
-

4. Adding a Button

Buttons perform actions when clicked.

```
import tkinter as tk


def say_hello():
    print("Hello Students!")


root = tk.Tk()

root.title("Button Example")


button = tk.Button(root, text="Click Me", command=say_hello)

button.pack(pady=10)


root.mainloop()
```

Explanation:

- `Button()` creates a button.
 - `command=say_hello` links the button to a function that runs when clicked.
-

5. Taking Input from the User

Use an **Entry box** to take text input.

```

import tkinter as tk

def show_input():
    name = entry.get() # Get text from entry box
    print("Hello", name)

root = tk.Tk()
root.title("Entry Example")

entry = tk.Entry(root)
entry.pack(pady=5)

button = tk.Button(root, text="Submit", command=show_input)
button.pack(pady=5)

root.mainloop()

```

Explanation:

- `Entry()` creates a single-line text box.
- `.get()` retrieves what the user typed.

6. Layout Managers (Arranging Widgets)

Tkinter offers three ways to arrange widgets:

Method	Description	Use
<code>pack()</code>	Stacks widgets vertically or horizontally	Simple layouts
<code>grid()</code>	Arranges widgets in rows and columns	Forms and tables
<code>place()</code>	Positions widgets at exact x, y coordinates	Precise control

Example using **grid**:

```

import tkinter as tk

root = tk.Tk()
root.title("Grid Example")

```

```
tk.Label(root, text="Name:").grid(row=0, column=0)
tk.Entry(root).grid(row=0, column=1)

tk.Label(root, text="Age:").grid(row=1, column=0)
tk.Entry(root).grid(row=1, column=1)

root.mainloop()
```

7. Message Boxes (Pop-ups)

Message boxes are small pop-up windows that display information or warnings.

```
import tkinter as tk

from tkinter import messagebox

def greet():
    messagebox.showinfo("Greeting", "Hello Everyone!")

root = tk.Tk()
root.title("Messagebox Example")

button = tk.Button(root, text="Click for Greeting", command=greet)
button.pack(pady=10)

root.mainloop()
```

Explanation:

- `messagebox.showinfo()` displays a pop-up with a title and message.
-

8. Simple Calculator Example

This mini-project performs addition of two numbers.

```
import tkinter as tk
```

```
def add_numbers():  
    n1 = int(entry1.get())  
    n2 = int(entry2.get())  
    result = n1 + n2  
    result_label.config(text="Result: " + str(result))  
  
root = tk.Tk()  
root.title("Mini Calculator")  
  
tk.Label(root, text="Enter First Number:").pack()  
entry1 = tk.Entry(root)  
entry1.pack()  
  
tk.Label(root, text="Enter Second Number:").pack()  
entry2 = tk.Entry(root)  
entry2.pack()  
  
tk.Button(root, text="Add", command=add_numbers).pack(pady=5)  
result_label = tk.Label(root, text="Result: ")  
result_label.pack()  
  
root.mainloop()
```

9. Changing Colors and Font

You can customize colors and fonts for widgets.

```
import tkinter as tk  
  
root = tk.Tk()  
root.title("Color Example")  
root.configure(bg="lightblue")
```

```
label = tk.Label(root, text="Colorful Label", bg="yellow", fg="red", font=("Arial", 16))
```

```
label.pack(pady=10)
```

```
root.mainloop()
```

Explanation:

- bg → background color
 - fg → text color
 - font=("FontName", size) sets font style.
-

10. Using Frames to Group Widgets

A **Frame** acts as a container to group multiple widgets.

```
import tkinter as tk
```

```
root = tk.Tk()
```

```
root.title("Frame Example")
```

```
frame = tk.Frame(root, bg="lightgray", bd=3)
```

```
frame.pack(padx=20, pady=20)
```

```
tk.Label(frame, text="This is inside the frame").pack()
```

```
tk.Button(frame, text="Click").pack()
```

```
root.mainloop()
```

Explanation:

- Frames help organize widgets neatly within sections.
-

11. Displaying Images

Tkinter can display images using `PhotoImage` or the `Pillow` library.

Example with PNG image:

```
import tkinter as tk
```

```
root = tk.Tk()

root.title("Image Example")


photo = tk.PhotoImage(file="photo.png")

label = tk.Label(root, image=photo)

label.image = photo # Prevent garbage collection

label.pack()


root.mainloop()
```

Example with JPG using Pillow:

```
from PIL import Image, ImageTk

import tkinter as tk
```

```
root = tk.Tk()

root.title("Image Example (JPG)")


img = Image.open("photo.jpg")

photo = ImageTk.PhotoImage(img)


label = tk.Label(root, image=photo)

label.image = photo

label.pack()


root.mainloop()
```

12. Handling Mouse and Keyboard Events

You can make your app respond when the user types or clicks.

```
import tkinter as tk


def on_click(event):

    print("Mouse clicked at:", event.x, event.y)
```

```
root = tk.Tk()

root.title("Event Example")


root.bind("<Button-1>", on_click) # Left mouse click

root.mainloop()
```

Explanation:

- `.bind("<Button-1>", function)` runs when the left mouse button is clicked.
 - The event object gives info like x, y coordinates.
-

13. Student Information Form Project

This small app takes user input and displays it back on screen.

```
import tkinter as tk
```

```
def show_info():

    name_val = name.get()

    age_val = age.get()

    class_val = cls.get()


    info = f"Name: {name_val}, Age: {age_val}, Class: {class_val}"

    result_label.config(text=info, fg="blue")
```

```
root = tk.Tk()

root.title("Student Information Form")

root.geometry("300x200")


tk.Label(root, text="Name:").grid(row=0, column=0, padx=5, pady=5)

name = tk.Entry(root)

name.grid(row=0, column=1, padx=5, pady=5)


tk.Label(root, text="Age:").grid(row=1, column=0, padx=5, pady=5)
```



```

age = tk.Entry(root)

age.grid(row=1, column=1, padx=5, pady=5)

tk.Label(root, text="Class:").grid(row=2, column=0, padx=5, pady=5)

cls = tk.Entry(root)

cls.grid(row=2, column=1, padx=5, pady=5)

tk.Button(root, text="Submit", command=show_info).grid(row=3, column=0, colspan=2, pady=10)

result_label = tk.Label(root, text="")

result_label.grid(row=4, column=0, colspan=2)

root.mainloop()

```

Explanation:

- Takes name, age, and class as input.
- Displays all entered details when “Submit” is clicked.

14. Summary Table

Purpose	Widget / Function	Example
Text display	Label	Label(root, text="Hi")
Input box	Entry	Entry(root)
Button	Button	Button(root, text="Click")
Pop-up	messagebox	messagebox.showinfo()
Image	PhotoImage / ImageTk	Label(root, image=photo)
Layout	pack, grid, place	widget.grid()
Grouping	Frame	Frame(root)
Event	.bind()	root.bind("<Key>", func)
